Programación de Sistemas y Dispositivos

Grado en Ingeniería Informática

Kernel Objects: *Semaphores*
Mutex and Message Queues

Prof. Jose Antonio Huerga

Kernel Objects: Semaphores

Los **semáforos** son, en software, mecanismos de bloqueo, sincronización y control de recursos que son compartidos en la ejecución de aplicaciones paralelas (En nuestro caso RTOSs que funcionan bajo una única CPU)

En µC-OS-III, disponemos de dos tipos de semáforos:

- Semáforos binarios (***Binary Semaphores***). Los posibles valores que puede tomar son `1 y 0.`

- Semáforos contadores (***Counting Semaphores***). Puede tomar valores los siguientes valores:
  - `0-255` (para sistemas de 8-bit)
  - `0-65535` (para sistemas de 16-bit)
  - `0-4292967295` (para sistemas de 32-bit)

Kernel Objects: Semaphores

Los **semáforos** pertenecen a los objetos del kernel, y en µC-OS-III se definen por la estructura **OS_SEM,** y que puede hallarse en el archivo header **os.h** (Del mismo tipo que **OS_TCB** para definir Tasks)

Function Name	Operation
OSSemCreate()	Create a semaphore.
OSSemDel()	Delete a semaphore.
OSSemPend()	Wait on a semaphore.
OSSemPendAbort()	Abort the wait on a semaphore.
OSSemPost()	Release or signal a semaphore.
OSSemSet()	Force the semaphore count to a desired value.

Kernel Objects: Semaphores

A la hora de crear un semáforo, se debe indicar el nombre que deseamos darle. Además, al igual que cuando creamos *Tasks*, debemos añadirlas a la estructura de datos que configura los valores por defecto. De esta manera, permite al sistema operativo saber que la variable asignada corresponde a un semáforo.

Se puede crear el número de semáforos que se desee, siempre y cuando no se supere la cantidad de memoria RAM disponible.

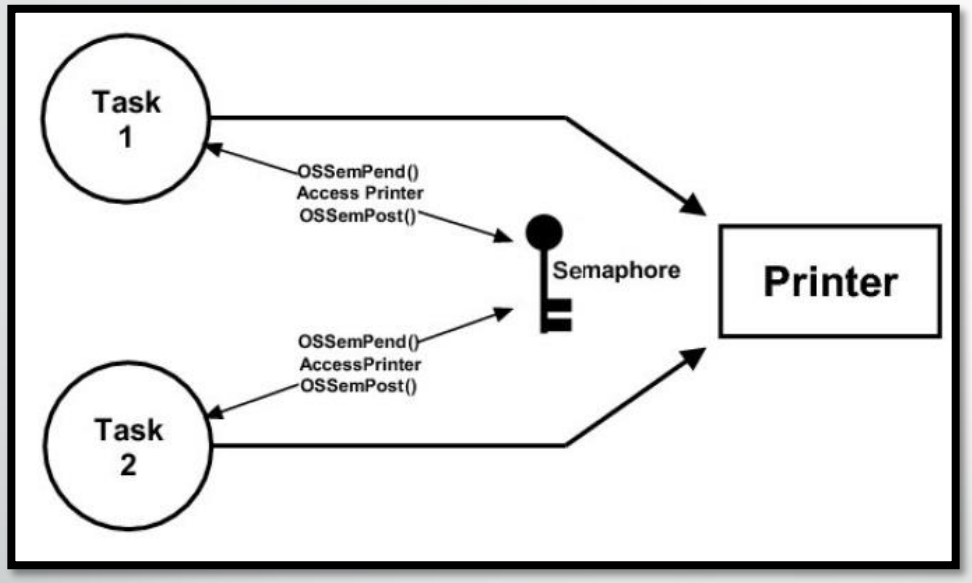
```
// Declarar semaforos
OS_SEM MySemaphore;
```

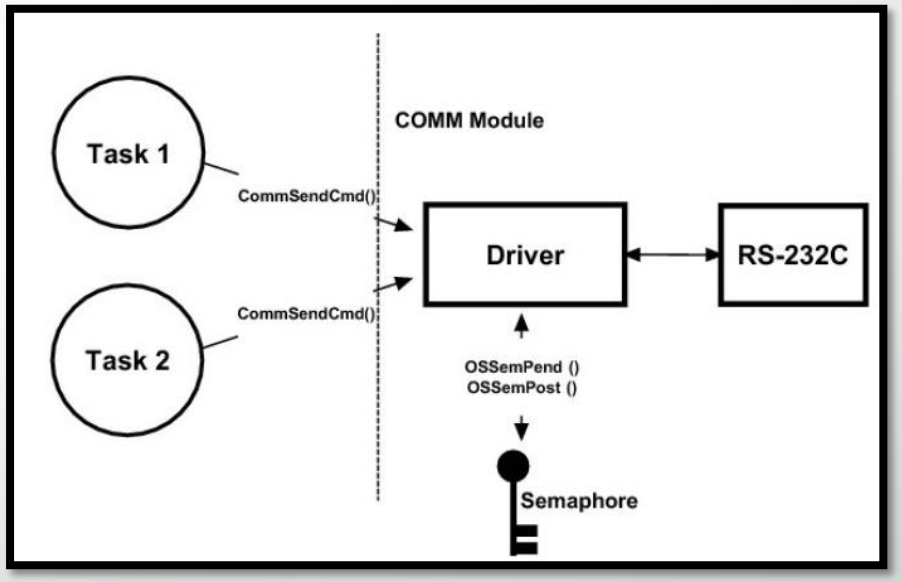
```
OS_ERR os_err;



OSSemCreate ((OS_SEM    *) &MySemaphore,
             (CPU_CHAR  *) "Semaforo de sincronizacion",
             (OS_SEM_CTR ) 0,    // valor inicial (1 = Disponible)
             (OS_ERR    *) &os_err);
```

Kernel Objects: Semaphores





Kernel Objects: Semaphores

Cuando deseamos liberar el recurso compartido, o informar de que está libre para otras Tasks, debemos indicarlo mediante la función **OSSemPost().** Por otro lado, para mantenerse a la "espera" de la liberación del semáforo, usaremos la función **OSSemPend()** en la Task que corresponda. Esto se muestra en las siguientes líneas de código. Como nota adicional, OSSemPost() se puede ejecutar dentro de las ISRs también (No es el caso de OSSemPend())

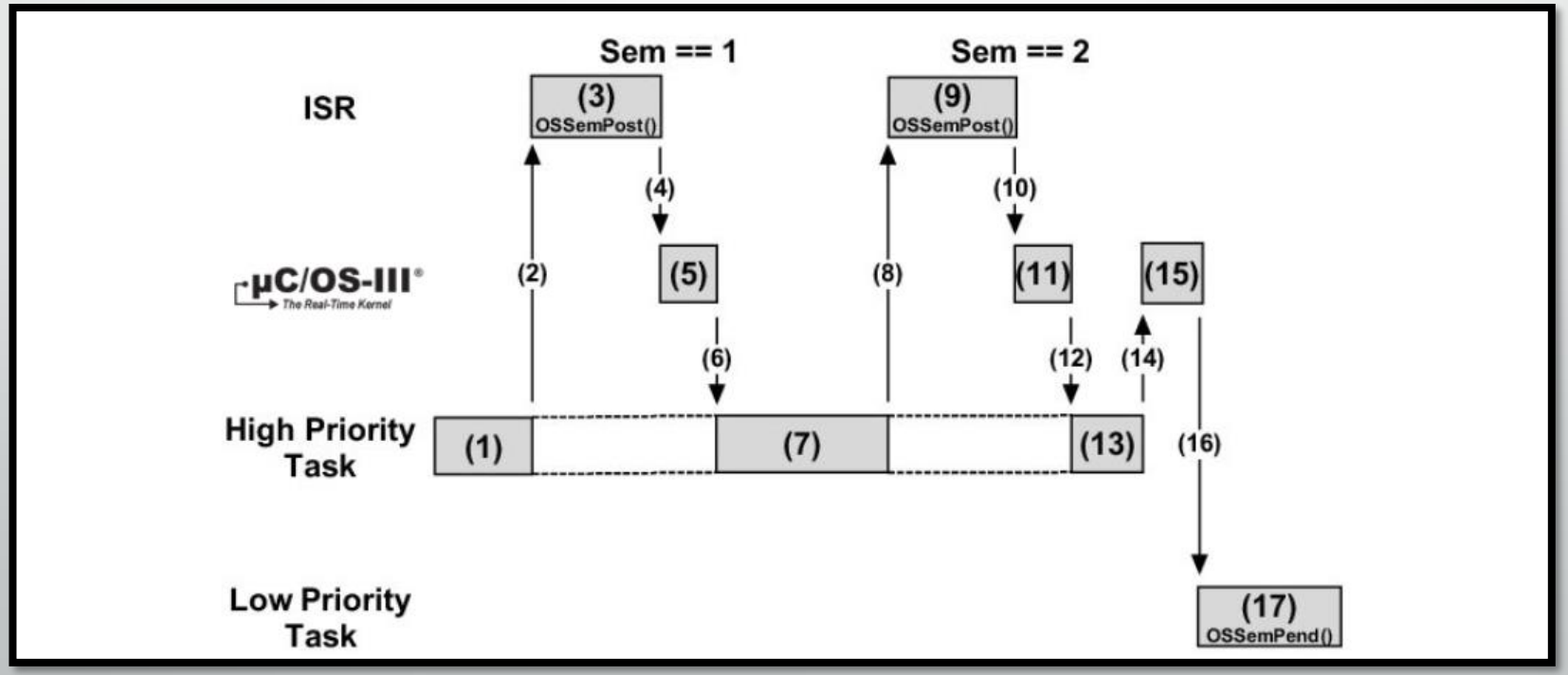
```
OSSemPend    (&MySemaphore,          // Esperamos por el semaforo para esetar disponible
              0,                     // No timeout
              OS_OPT_PEND_BLOCKING,  // Esperar hasta que el semaforo sea liberado o el timeout expire
              &ts,                   // timestamp (para saber cuando se ha liberado el recurso  (OS_TS_GET()))
              &os_err);
```

```
OSSemPost    (&MySemaphore, // Liberar recurso compartido (semaforo = 1)
              OS_OPT_POST_1,
              &os_err);
```

Kernel Objects: Semaphores

CREDIT TRACKING

µC-OS-III permite llevar un "conteo" de las veces que se ha liberado o posteado un semáforo, antes de que una Task haya podido tomarlo mediante `OSSemPend()`. El valor del conteo equivaldrá al número de veces que la tarea que espera por el semáforo pueda ejecutarse de manera continua, siempre que sea la tarea más prioritaria.



Priority Inversión con Semáforos

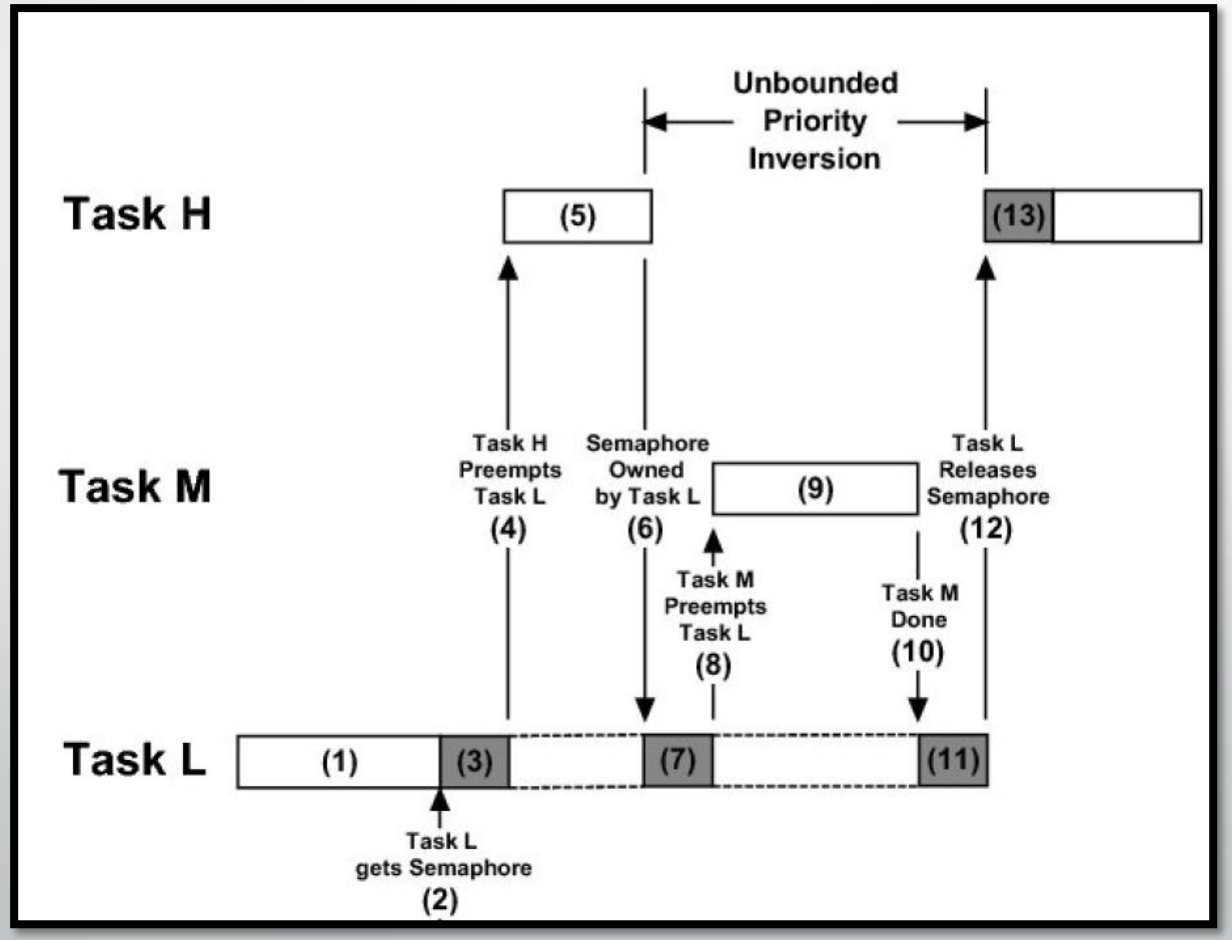
El fenómeno de inversión de prioridades se produce cuando una tarea de **alta prioridad** se comporta como una de **baja prioridad**. Esto es debido a que, tanto una tarea de baja prioridad como una de alta prioridad, desean acceder al mismo recurso.

Si la tarea de baja prioridad consigue acceder antes a este recurso y acto seguido una tarea de alta prioridad desea ejecutarse y tomar también el control de dicho recurso compartido, deberá esperar a que éste sea liberado (por parte de la tarea de baja prioridad).

En función de la duración de otros procesos de mediana prioridad, la inversión de prioridades se producirá durante un tiempo más o menos extenso. Este tiempo se denomina, en la literatura de sistemas operativos en tiempo real, como "**Unbounded Priority Inversion**".

Priority Inversión con Semáforos

EJEMPLO PRÁCTICO



Kernel Objects: Mutex

El **Mutex** es uno de los objetos de kernel en RTOS más usados. Tiene funcionalidades similares a las del semáforo, siendo particularmente usado para el control de recursos compartidos dentro de una aplicación.

Como sabemos, los semáforos pueden usarse como sincronizadores y como medio para el control de accesos indebidos a recursos que pueden compartirse en un mismo proceso. En este último caso, el semáforo puede producir el indeseado fenómeno de **inversión de prioridad**, que en función de la carga de trabajo del RTOS, podrá ser más o menos severo.

No existe un método para resolver completamente este problema, sin embargo, se pueden reducir sus efectos. **Para esta tarea en particular surgió el Mutex**.

Kernel Objects: Mutex

El **Mutex** se aplica del mismo modo que los semáforos. En el código se deben seguir los siguientes pasos:

1. Esperar a la liberación del recurso compartido. Esta operación se realiza mediante **`OSMutexPend()`**.
2. Una vez liberado, se procede a acceder al recurso compartido (e.g Uart, SPI, canal ADC, etc).
3. Después de su uso, se debe liberar el recurso compartido. Esto se realiza mediante **`OSMutexPost()`**.

Las funciones `OSMutexPend()` y `OSMutexPost()` solo pueden llamarse desde Tasks. En este caso no pueden usarse dentro de las ISRs.

Mutex: solución a la inversión de prioridades

Volviendo al problema de la inversión de prioridades, repasaremos ahora lo que ocurre con un Mutex:

Primeramente, una tarea de baja prioridad accede al recurso compartido. Acto seguido, una tarea de alta prioridad desea acceder a dicho recurso. Sin embargo, sabe que está siendo usado por otra tarea. Aquí, el Mutex sabe que la tarea que actualmente dispone del control del recurso posee una prioridad menor que la tarea que desea acceder al mismo.

En este caso, ahora el Mutex produce en el sistema una **herencia de prioridades temporal**, de modo que la tarea de baja prioridad ahora toma el grado de prioridad de la tarea que deseaba acceder al recurso compartido. De esta manera, la tarea, que en un principio era de baja prioridad, ahora puede "despachar" rápidamente el recurso compartido (ya que ha heredado una prioridad mayor), para así poder entregárselo a la verdadera tarea de mayor prioridad a la mayor brevedad.

Mutex: solución a la inversión de prioridades

Es importante denotar que, al heredar la tarea de baja prioridad una prioridad mayor, **se impide que otras tareas de prioridad media puedan ejecutarse y alargar el proceso**.

Esto es importante, ya que son las tareas de "mediana" prioridad (aquellas que se encuentran entre nuestra tarea de alta prioridad y la tarea de baja prioridad) las que determinan la severidad del fenómeno de prioridad de inversión, cuando este es producido haciendo uso de los semáforos.

Kernel Objects: Mutex

Funciones propias relativas al Mutex

Function Name	Operation
`OSMutexCreate()`	Create a mutex.
`OSMutexDel()`	Delete a mutex.
`OSMutexPend()`	Wait on a mutex.
`OSMutexPendAbort()`	Abort the wait on a mutex.
`OSMutexPost()`	Release a mutex.

Kernel Objects: Mutex

Funciones más frecuentes. Ejemplo en MPLAB

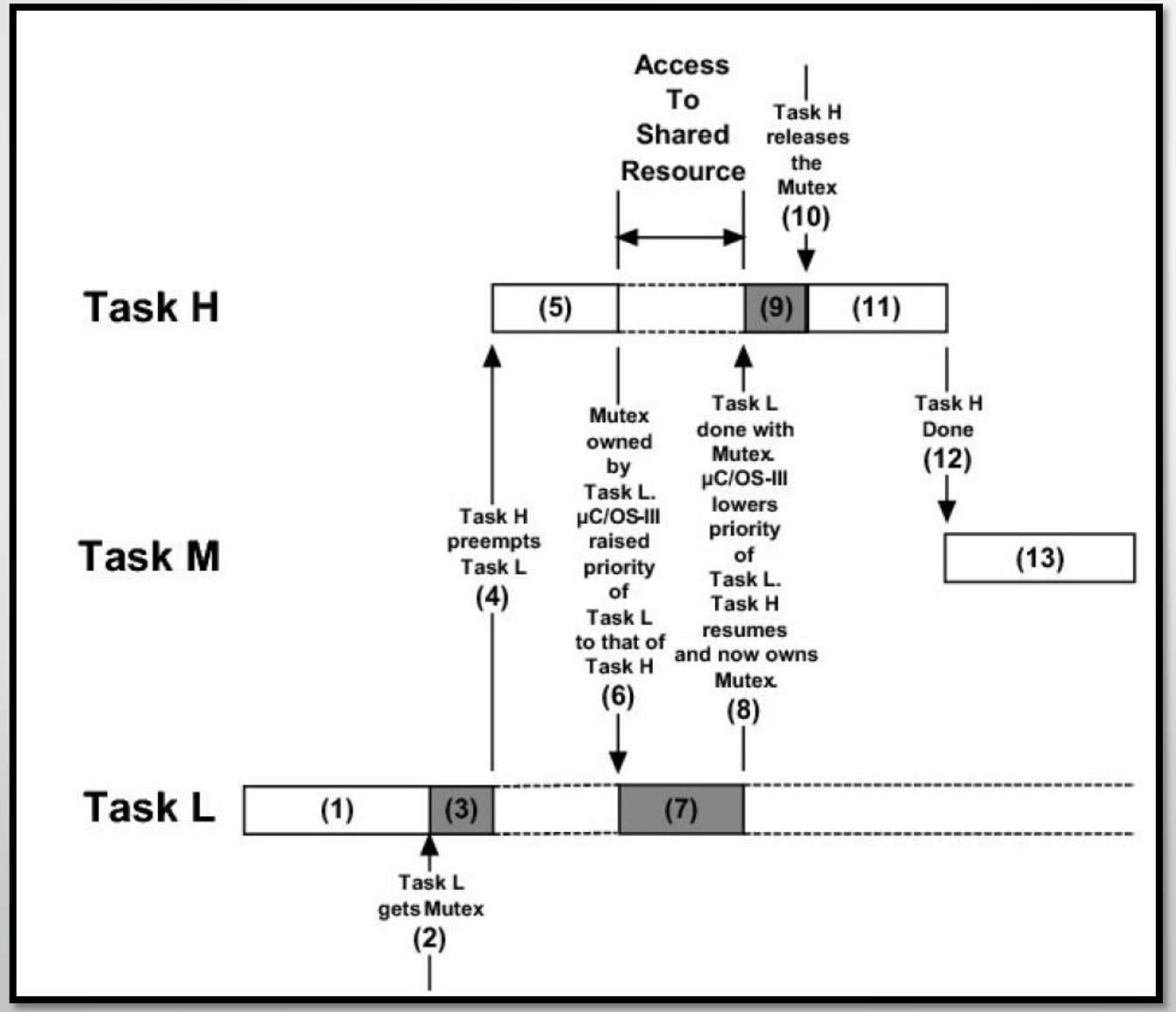
```
// Declarar Mutexes
OS_MUTEX MyMutex;
```

```
OSMutexCreate ((OS_MUTEX *) &MyMutex,
               (CPU_CHAR  *) "Mutex para control de recursos compartidos",
               (OS_ERR    *) &os_err);
```

```
OSMutexPend((OS_MUTEX     *) &MyMutex,
            (OS_TICK       ) 0,
            (OS_OPT        ) OS_OPT_PEND_BLOCKING,
            (CPU_TS       *) &ts,
            (OS_ERR       *) &os_err);
```

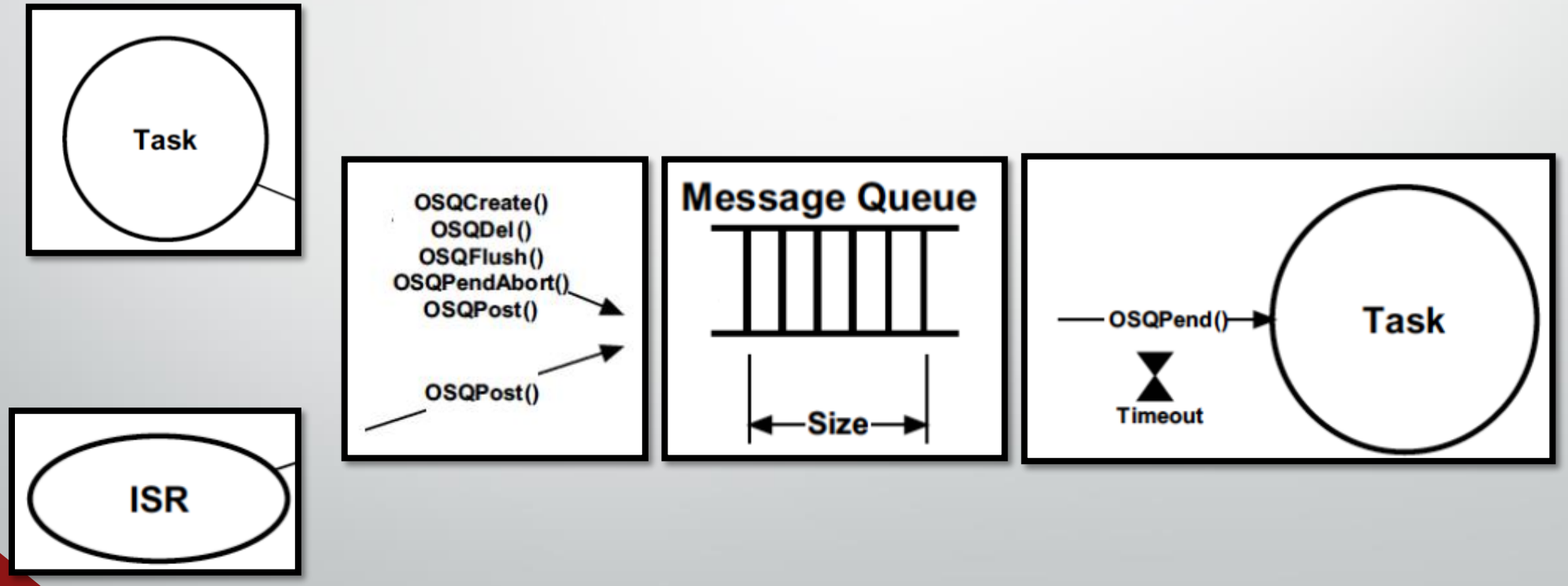
```
OSMutexPost((OS_MUTEX *) &MyMutex,
            (OS_OPT    ) OS_OPT_POST_NONE,
            (OS_ERR   *) &os_err);
```

Mutex: solución a la inversión de prioridades



Kernel Objects: Message Queue

Este objeto de Kernel es usado para realizar comunicaciones entre Tareas (*inter-task communication*), o bien entre ISRs y Tareas. Además, puede ser usado como método de sincronización (de la misma manera que los semáforos), con la ventaja de que además se pueden transmitir y recibir mensajes.



Kernel Objects: Message Queue

Como veremos más adelante en detalle, tenemos varios tipos de buffers de mensajes con los que podemos operar en µC-OS-III. Por un lado tenemos los buffers propios o individuales que cada Task dispone. Por otro lado, podemos crear buffers que denominaremos "externos", ya que no son inherentes a cada tarea, y son creados por el usuario. Normalmente estos últimos se usan para comunicaciones que involucran más de dos Tasks (***Broadcasting***).

Los buffers individuales se usan cuando queremos comunicar datos con una tarea en particular. Por ejemplo cuando una ISR comunica datos con una tarea, o cuando una tarea únicamente se debe comunicar con otra. Este método requiere menos trabajo por parte del usuario, ya que no tiene que crear los buffers (ya se encuentran creados a la hora de crear la propia tarea dentro de nuestro `main()`).

Para diferenciar cuando usar un buffer u otro, µC-OS-III diferencia las funciones de la siguiente manera:

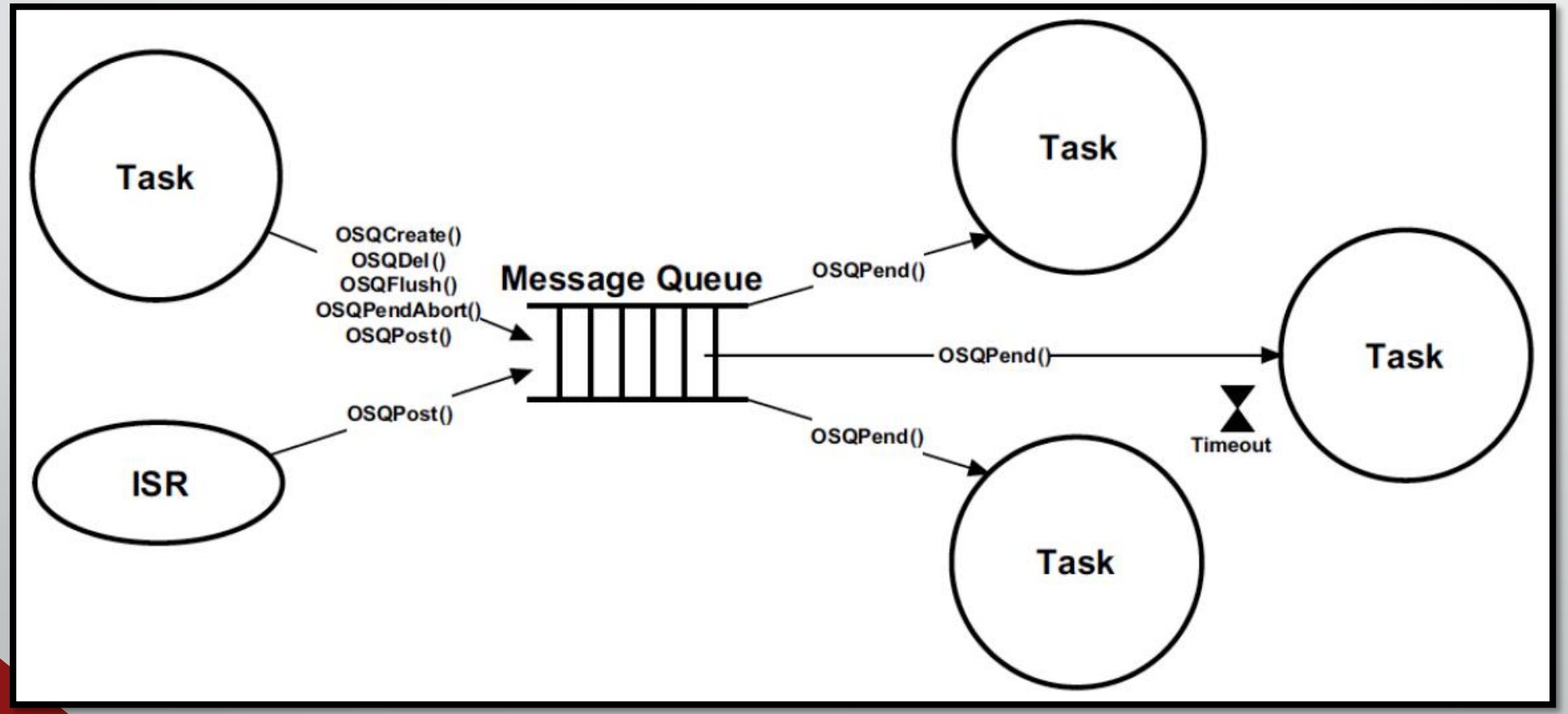
Para los buffers individuales, se añade la palabra "Task" dentro del nombre de la función, mientras que el buffer externo no lo lleva. Ejemplo:

`OSTaskQPend()` (Función de espera para buffers internos).

`OSQPend()` (Función de espera para buffers externos).

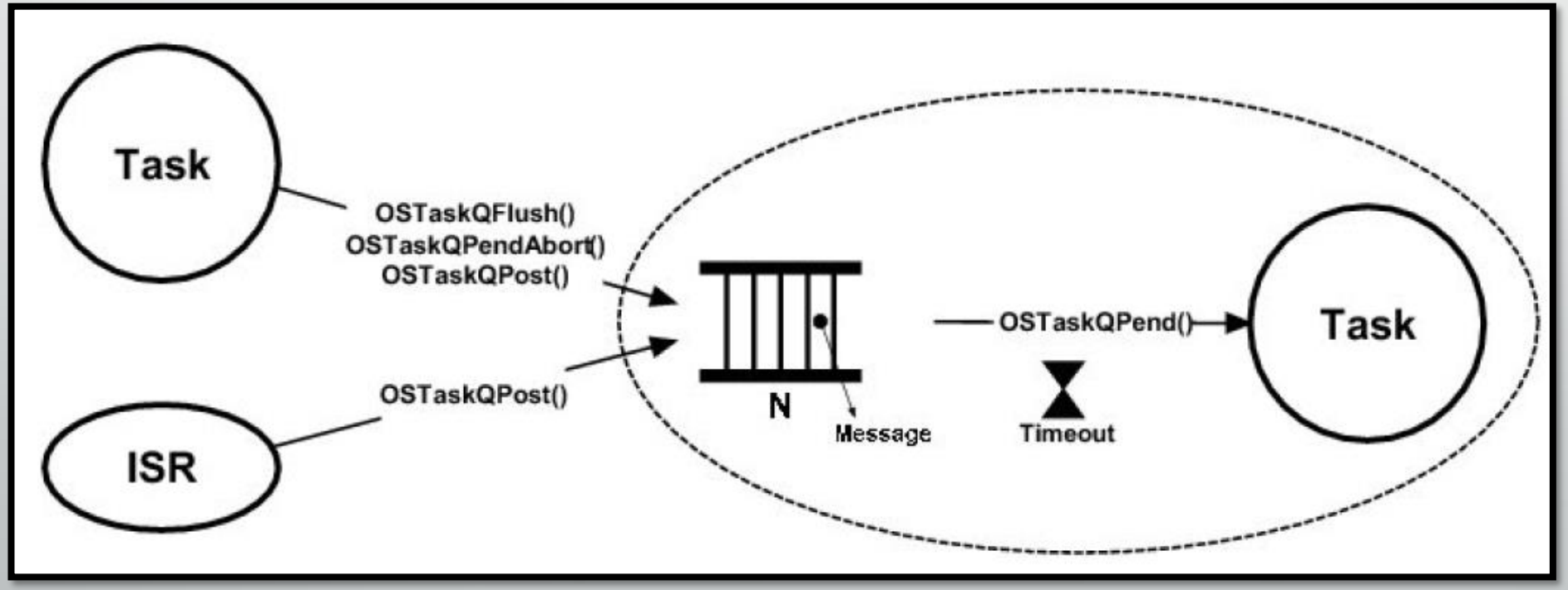
Kernel Objects: Message Queue

Broadcasting: En este caso, la tarea (o ISR) encargada de transmitir un mensaje a la cola de mensajes, se encuentra a la espera de ser recibido por varias tareas de la aplicación. Fijaos como las funciones usadas corresponden a buffer de mensajes externo.



Kernel Objects: Message Queue

Task Message Queue: Puesto que es extraño encontrar tareas que se encuentren esperando por un mismo mensaje que debe ser enviado a una cola de mensajes externa común (compartida por varias tareas), µC-OS-III incorpora un propio buffer de mensajes, que todas las tareas incluyen a modo de cola de mensajes individual, tal y como hemos visto anteriormente.



Kernel Objects: Message Queue

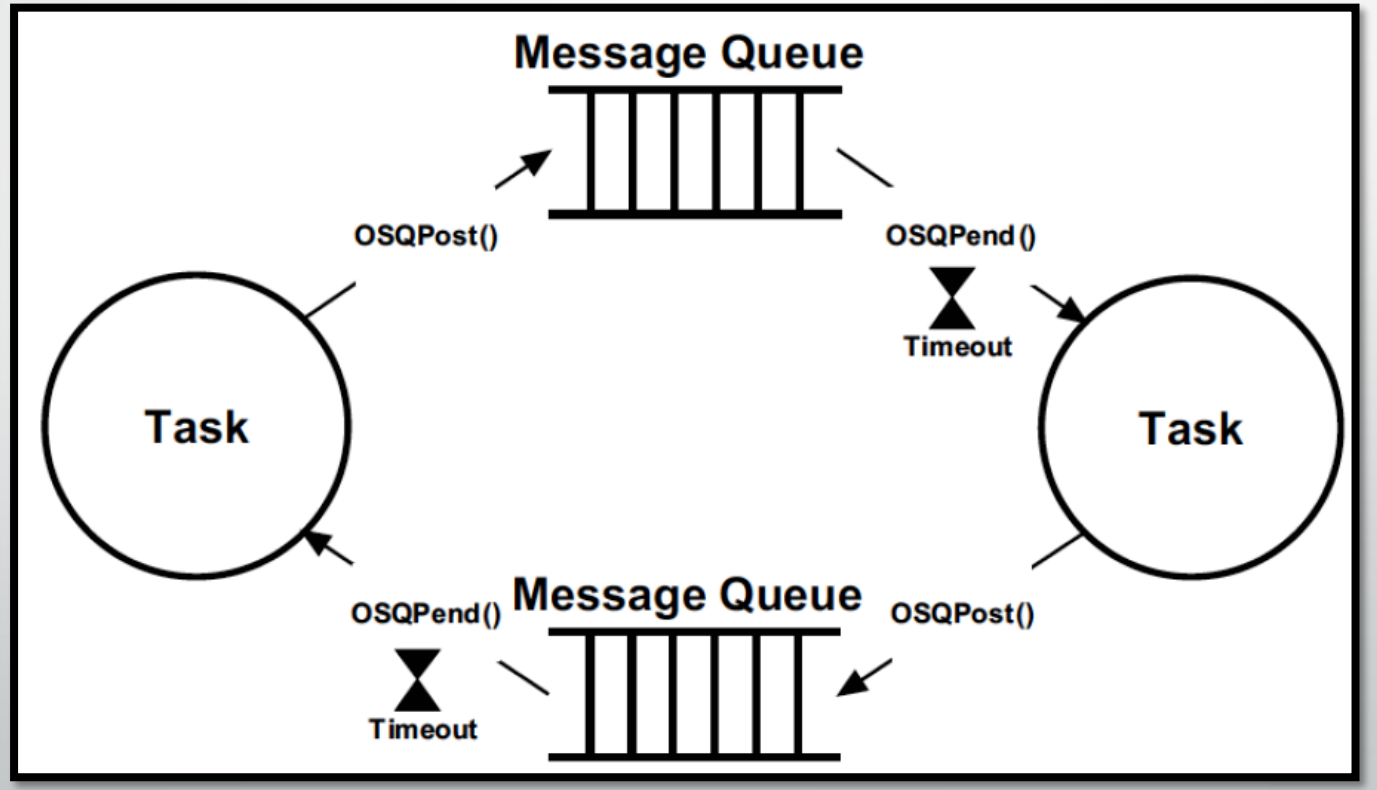
Task Message Queue: Para poder habilitar este servicio de buffer interno, que por defecto puede estar deshabilitado, se tiene que definir algunos parámetros en el header **`os_cfg.h`**

Dentro de `os_cfg.h` nos dirigiremos a la siguiente instrucción, para habilitar las colas de mensajes individuales de cada tarea

OS_CFG_TASK_Q_EN	to	**DEF_ENABLED (1u)**	*Habilitamos Built-in Task Message*
OS_CFG_TASK_Q_EN	to	**DEF_DISABLED (0u)**	*Deshabilitamos Built-in Task Message*

Kernel Objects: Message Queue

Bilateral Rendez-vous (Sincronización bilateral): Permite sincronizar las actividades de dos tareas entre si. Cada cola de mensajes almacena como máximo un **único** mensaje.



Kernel Objects: Message Queue

Funciones propias para configurar colas de mensajes externas (External Message Queue)

Function Name	Operation
`OSQCreate()`	Create a message queue.
`OSQDel()`	Delete a message queue.
`OSQFlush()`	Empty the message queue.
`OSQPend()`	Wait for a message.
`OSQPendAbort()`	Abort waiting for a message.
`OSQPost()`	Send a message through a message queue.

Kernel Objects: Message Queue

Funciones propias para configurar colas de mensajes internas (Built-in Message Queue)

Function Name	Operation
`OSTaskQPend()`	Wait for a message.
`OSTaskQPendAbort()`	Abort the wait for a message.
`OSTaskQPost()`	Send a message to a task.
`OSTaskQFlush()`	Empty the message queue.

Kernel Objects: Message Queue

Funciones más frecuentes. Ejemplo en MPLAB para buffer externo

```
OSQCreate    ((OS_Q      *)&MyExternalQueue,
              (CPU_CHAR   *)"My Queue",
              (OS_MSG_QTY )20,            // Numero máximo de mensajes permitidos
              (OS_ERR     *)&os_err);
```

```
var_rec = OSQPend                    (&MyExternalQueue,
                                      0,
                                      OS_OPT_PEND_BLOCKING,
                                      &prueba_size,
                                      &ts,
                                      &os_err);
```

```
// Declarar Colas de Mensajes
OS_Q MyExternalQueue;
```

```
OSQPost          (&MyExternalQueue,
                  (void *)(unsigned int)(var_char),
                  sizeof(var_char),
                  OS_OPT_POST_FIFO,
                  &os_err);
```

Tanto ISRs como Tasks pueden usar la función de posteado de mensajes (Post). Sin embargo, esperar por el mensaje solo es aplicable a tareas (Pend).

Kernel Objects: Message Queue

EJEMPLO: Transmitir variables mediante cola de mensajes externa

Existen varias formas de transmitir un mensaje a través de Message Queues. Las más utilizadas en µC-OS-III son:

1. Transmitiendo el propio contenido de la variable (se realiza una copia de su contenido dentro del buffer). Esta es la opción más segura cuando trabajamos con aplicaciones paralelas o con ISRs. La desventaja es que se usa más memoria RAM para almacenar el contenido de la copia.

2. Transmitiendo la dirección de la variable (se comparte la dirección y por tanto el contenido puede ser accedido por más de una aplicación a la vez, por parte de una ISR o Task. La ventaja es que es más eficiente en términos de gasto de memoria. Sin embargo, al haberse compartido la misma dirección de memoria, varios procesos de la aplicación pueden acceder al mismo contenido a la vez. Supóngase el caso en el que una Task se encuentra accediendo al contenido de una variable, y justo en ese proceso una ISR accede a ese contenido para actualizar su valor. En este caso, la Task no tiene constancia de que otro proceso está accediendo a la misma variable, y esto puede generar contenido corrupto.

Kernel Objects: Message Queue

EJEMPLO: Transmitir variables mediante buffer de mensajes externo

Como ejemplo, supóngase que una Task desea acceder al contendido de una variable "*var*", que contiene el valor `0xAAAAFFFF` `(32-bit)`

1. Task consigue leer los 2 primeros bytes de la variable, antes de que una ISR haya saltado. (**FFFF**).
2. La ISR salta, y actualiza el valor de "*var*" a `0xDDDD0000`.
3. Una vez acabado el proceso de la ISR, el kernel restaura los contenidos de status, CPU y contador del programa para volver a situarse donde estaba.
4. Task continua accediendo al valor de "*var*" para leer los 2 últimos bytes, y así obtener el resultado final: `0xDDDDFFFF`.

Como puede verse, el resultado ha cambiado y no se ha obtenido el valor real actualizado. Por esta razón, es importante tener control de acceso a las variables que pueden ser accedidas en cualquier momento por otros procesos. Para estas situaciones se han creado los **Message Queues**, además de que son capaces de cambiar el estado de las tareas y añadir prioridad al sistema.